ForgeAI

Project Specification Document

AMD AI DevMaster Hackathon -- Track 2

Team: Script-Kitty01 | Date: August 6, 2026

1. Application Scenarios

ForgeAI is a conversational multi-agent AI engineering assistant that brings the power of a full engineering review team to any developer's local machine. It runs entirely on AMD Radeon GPUs via ROCm, ensuring complete data privacy -- no code ever leaves the user's machine.

Target Users

User	Scenario
Solo Developers	Get instant code review feedback without waiting for teammates
Small Teams	Automated first-pass review before human PR review
Security-Conscious Orgs	Private vulnerability scanning without sending code to cloud APIs
Open Source Maintainers	Triage incoming PRs with automated quality checks
Students & Learners	Learn best practices through AI-guided code analysis

Key Use Cases

- Security Audit -- Find vulnerabilities: hardcoded secrets, SQL injection, command injection, weak hashing
- Performance Review -- Identify O(n^2) loops, memory-heavy patterns, blocking I/O, unnecessary deep copies
- Architecture Assessment -- Evaluate SOLID principles, coupling/cohesion, god classes, circular dependencies
- DevOps Readiness -- Check Dockerfile quality, hardcoded configs, localhost references, missing health checks
- Full Codebase Review -- All 4 agents run, Consensus agent synthesizes findings with quality score

2. Agent Architecture

ForgeAI uses a 6-agent collaborative system orchestrated by LangGraph, with all inference running locally on AMD Radeon GPU via ROCm.

Agent Roles

Agent	Role	Tools	Output
Planner	Orchestrates analysis, decomposes queries	--	Structured task plan (JSON)
Security	Finds vulnerabilities (OWASP, CWE, secrets)	Bandit, Semgrep	Severity-scored findings
Performance	Spots bottlenecks & anti-patterns	Code Search, Pattern Match	Performance findings
Architecture	Evaluates design & modularity	Git Analyzer, Code Search	Architecture findings
DevOps	Checks deployment readiness	Dockerfile Validator	DevOps findings
Consensus	Cross-review debate & final verdict	--	Unified report + quality score

Workflow

User Query -> Planner -> RAG Retrieval -> [Security, Perf, Arch, DevOps] in parallel -> Consensus (debate + synthesis) -> Formatted Response -> User

3. Core Capabilities

3.1 Multi-Agent Collaboration

- 6 specialized agents with distinct system prompts and expertise domains
- Cross-review debate: Consensus agent identifies conflicting findings and facilitates resolution
- Confidence scoring: Each finding includes a 0.0-1.0 confidence score, adjusted during debate

3.2 Retrieval-Augmented Generation (RAG)

- ChromaDB vector store with persistent local storage
- sentence-transformers embeddings computed on ROCm GPU
- Code files chunked by function/class boundaries for semantic retrieval

3.3 Tool Invocation

- Bandit -- Python static security analysis
- Semgrep -- Multi-language pattern-based scanning
- Git Analyzer -- Churn hotspots, commit history analysis
- Dockerfile Validator -- Best practices checking
- Code Search -- Regex-based pattern matching across codebase

3.4 Conversation Memory

- LangGraph checkpointing with MemorySaver
- Full conversation history preserved across turns
- Multi-turn drill-down: 'Tell me more about that SQL injection'

3.5 Privacy-First Design

- 100% local inference -- no data sent to external APIs
- All models run on AMD Radeon GPU via ROCm
- No telemetry, no cloud dependency

4. Model Introduction & Local Deployment Plan

4.1 LLM: Llama 3.2 3B Instruct (Q4_K_M Quantized)

Property	Value
Base Model	Meta Llama 3.2 3B Instruct
Quantization	Q4_K_M (4-bit with medium quality)
Format	GGUF
Size on Disk	~2.0 GB
VRAM Usage	~2.6 GB
Context Window	2048 tokens
Batch Size	512 tokens
Inference Engine	llama-cpp-python with HIP/ROCm backend

4.2 Embedding Model

Property	Value
Model	all-MiniLM-L6-v2

Dimension	384
GPU Backend	ROCm via PyTorch

4.3 Quick Start

```
git clone https://github.com/Script-Kitty01/devMaster_amd.git
cd devMaster_amd
pip install -e .
# Place Llama-3.2-3B-Instruct-Q4_K_M.gguf in models/
python -m src.main
```

5. AMD Radeon GPU Optimization

5.1 HIP/ROCm Build

```
cmake .. -DGGML_HIP=ON -DAMDGPU_TARGETS=gfx1100
  -DCMAKE_BUILD_TYPE=Release -DBUILD_SHARED_LIBS=ON
  -DGGML_HIP_GRAPHS=ON -DGGML_HIP_MMQ_MFMA=ON
```

5.2 Performance Benchmarks

Metric	Value
Model Load Time	575 ms
Prompt Eval	57.51 ms/token (17.39 tok/s)
Token Generation	8.05 ms/token (124.25 tok/s)
VRAM Usage	2.57 GB / 51 GB
ROCm Compute Buffer	256.5 MiB

5.3 CPU vs GPU Comparison

Metric	CPU Only	GPU (ROCm/HIP)	Speedup
Token Generation	~8 tok/s	124 tok/s	15.5x
Model Load	~2.5s	0.58s	4.3x
Embedding Gen	~50 docs/s	~400 docs/s	8x

6. Technical Stack

Layer	Technology
GPU Runtime	AMD ROCm 7.2.1, HIP 7.2.53211
Inference Engine	llama-cpp-python 0.3.34 (custom HIP build)
LLM	Llama 3.2 3B Instruct Q4_K_M GGUF
Embeddings	sentence-transformers (all-MiniLM-L6-v2) on ROCm PyTorch
Agent Framework	LangGraph (StateGraph + checkpointing)
RAG	ChromaDB (persistent local vector store)
UI	Streamlit
Tools	Bandit, Semgrep, GitPython, Docker
Language	Python 3.12

7. Innovation Highlights

- 6-Agent Collaborative System -- Full engineering team with specialized roles, cross-review debate, and consensus building
- 100% Local on AMD GPU -- Complete privacy; all inference, embeddings, and analysis run on the user's Radeon GPU
- Custom HIP Build -- llama.cpp compiled from source with GGML_HIP=ON, targeting gfx1100 with CUDA graphs and MFMA optimizations
- Structured Multi-Turn Memory -- LangGraph checkpointing preserves full conversation context, enabling deep drill-down analysis
- Tool-Augmented Agents -- Agents invoke real static analysis tools (Bandit, Semgrep) for evidence-based findings

Built for AMD AI DevMaster Hackathon 2026 -- Track 2